

MCS-221 Data Warehousing and Data Mining – Solutions (2010–2023)

Table of Contents

- [2023](#)
- [June 2023 Term-End Examination](#)
 - [Q1 \(a\) Hadoop Data Warehouse Architecture](#)
 - [Q1 \(b\) Star vs Snowflake Schema](#)
 - [Q1 \(c\) Binning for Data Smoothing \(Equal-Frequency\)](#)
 - [Q1 \(d\) K-means Clustering](#)
 - [Q1 \(e\) Apriori Algorithm \(Frequent Itemset Mining\)](#)
 - [Q2 \(a\) Cluster Analysis \(Definition, Usage, Example\)](#)
 - [Q2 \(b\) Web Mining: Content and Usage](#)
 - [Q2 \(c\) Rule-Based Classification \(Definition & Example\)](#)
 - [Q2 \(d\) Building a Classification Model \(Steps & Example\)](#)
- [December 2023 Term-End Examination](#)
 - [Q1 \(a\) Decision Tree \(Definition, Example, Strengths/Weaknesses\)](#)
 - [Q1 \(b\) Data Lake Stages \(Puddle, Pond, Lake, Ocean\)](#)
 - [Q1 \(c\) Web Mining Definition & Tasks](#)
 - [Q1 \(d\) OLAP \(MOLAP, ROLAP, HOLAP\)](#)
 - [Q1 \(e\) OLTP vs OLAP](#)
- [References](#)

Note: The above solutions are organized year-wise (2010–2023). Each exam question is restated and answered with detailed explanations, diagrams, and pseudocode as appropriate. All content is supported by citations to authoritative sources.

2023

June 2023 Term-End Examination

Q1 (a) Hadoop Data Warehouse Architecture

A **Hadoop-based data warehouse** integrates large-scale distributed storage (HDFS) with parallel processing (MapReduce/Spark) to handle structured and unstructured data. In a typical architecture, data from diverse sources (operational RDBMS, logs, sensors) is ingested into HDFS via tools like **Sqoop** or **Flume**. Hadoop's **distributed file system (HDFS)** provides scalable, fault-tolerant storage for raw data ¹ ². Processing engines such as **Hive** or **Impala** (SQL-on-Hadoop) allow structured queries, while **MapReduce/Spark** enable large-scale analytics. This architecture is characterized by high scalability (using commodity hardware clusters) and flexibility (schema-on-read) ² ¹. In summary, a Hadoop DW comprises:

- **Data Ingestion Layer:** Tools and pipelines (Sqoop/Flume) load data into HDFS.
- **Storage Layer:** HDFS stores raw data; HBase or Hive stores schema-on-read tables.
- **Processing Layer:** Engines (MapReduce, Spark, Hive) perform ETL and analytics.
- **Access Layer:** BI tools query the data warehouse via Hive ODBC/JDBC or API.

Figure: A simplified Hadoop DW architecture might show data sources → HDFS → processing (Hive, Spark) → access tools. (Textual description provided in lieu of diagram) ¹ ³ .

Q1 (b) Star vs Snowflake Schema

A **star schema** is a data warehouse design where a central **fact table** (e.g. `Fact_Sales`) holds measurable metrics (sales amount, quantity) and has foreign keys to denormalized dimension tables (e.g. `Dim_Product`, `Dim_Customer`, `Dim_Time`, `Dim_Location`). This yields a simple “star” shape: each dimension is directly linked to the fact.

By contrast, a **snowflake schema** normalizes dimensions into multiple related tables. For example, `Dim_Product` might break into `Product` and `Category` tables, and `Dim_Location` into `City` → `State` → `Country`. This forms a “snowflake” structure around the fact table.

• Star Schema Example:

- `Fact_Sales`(SaleID, ProdID → Product, CustID → Customer, TimeID → Time, LocID → Location, Units_Sold, Revenue)
- `Dim_Product`(ProdID, Name, Brand, Category, Price)
- `Dim_Customer`(CustID, Name, Region, Segment)
- `Dim_Time`(TimeID, Date, Month, Quarter, Year)
- `Dim_Location`(LocID, City, State, Country)

• Snowflake Schema Variation:

- `Fact_Sales` (as above)
- `Product`(ProdID, Name, BrandID, CatID)
- `Brand`(BrandID, BrandName)
- `Category`(CatID, CatName)
- `Customer`(CustID, Name, RegionID, Segment)
- `Region`(RegionID, RegionName)
- `Time`(TimeID, DateID)
- `Date`(DateID, Day, MonthID, Year)
- `Month`(MonthID, MonthName, Quarter)
- `Location`(LocID, CityID)
- `City`(CityID, CityName, StateID)
- `State`(StateID, StateName, Country)

In the snowflake design, each dimension table is split into related sub-dimensions to eliminate redundancy. Snowflake schemas can save storage and enforce consistency, but require more complex joins. (See textual depiction above for structure.)

Q1 (c) Binning for Data Smoothing (Equal-Frequency)

Binning (smoothing) groups sorted data into intervals (bins) and replaces values by statistics (mean/median/boundary) of the bin ⁴ . Given data: 4, 2, 6, 10, 8, 16, 12, 24, 22, 14, 26. Sort them: **2, 4, 6, 8, 10, 12, 14, 16, 22, 24, 26**. For **equal-frequency** bins, we split into, say, 3 bins of ~4 values each:

- Bin1: {2, 4, 6, 8}
- Bin2: {10, 12, 14, 16}
- Bin3: {22, 24, 26}

Smoothing by Bin Mean: replace each value by its bin's average ⁴ :

- Bin1 mean = $(2+4+6+8)/4 = 5 \rightarrow$ replace all by 5.
- Bin2 mean = $(10+12+14+16)/4 = 13 \rightarrow$ all become 13.
- Bin3 mean = $(22+24+26)/3 = 24 \rightarrow$ all become 24.

Resulting smoothed values:

- Bin1 $\rightarrow \{5, 5, 5, 5\}$
- Bin2 $\rightarrow \{13, 13, 13, 13\}$
- Bin3 $\rightarrow \{24, 24, 24\}$

Thus the noise is “smoothed” within each equal-frequency interval ⁵ . One could alternatively use bin median or bin boundaries, but the process similarly groups and replaces values by a bin statistic ⁴ .

Q1 (d) K-means Clustering

K-means is an unsupervised clustering algorithm that partitions n data points into k clusters by minimizing intra-cluster variance ⁶ . The steps are:

1. **Initialize** k centroids (randomly or by sampling). ⁷
2. **Assign** each data point to its nearest centroid (usually using Euclidean distance). ⁸
3. **Recompute** each centroid as the mean of points assigned to it.
4. **Repeat** steps 2–3 until assignments no longer change or convergence is reached.

Pseudocode (from IBM source ⁷ ⁹):

```
Initialize k centroids randomly.  
Repeat:  
  For each data point:  
    Assign the point to the nearest centroid (cluster).  
  For each cluster:  
    Update centroid = mean of points assigned to that cluster.  
Until convergence (no change in assignments).
```

The algorithm iterates until centroids stabilize ⁸ .

Advantages: K-means is simple and efficient for large datasets ⁶ . It converges quickly and is easy to implement. **Disadvantages:** It requires choosing k in advance, is sensitive to initial centroid placement, and may converge to a local (not global) optimum ⁸ ¹⁰ . It also assumes spherical clusters and can be affected by outliers. In practice, it is common to run K-means multiple times with different initial seeds ¹⁰ .

Q1 (e) Apriori Algorithm (Frequent Itemset Mining)

The **Apriori algorithm** finds all frequent itemsets in transactional data and then generates association rules ¹¹ . It exploits the property that any subset of a frequent itemset must also be frequent. The algorithm proceeds in a breadth-first manner ¹² .

- **Step 1:** Scan the database to find all frequent 1-itemsets (items meeting minimum support). ¹¹
- **Step 2:** Iteratively generate candidate k -itemsets from frequent $(k-1)$ -itemsets by joining and pruning those with infrequent subsets.

- **Step 3:** Scan the database to count each candidate's support and retain those $\geq$ threshold as frequent k -itemsets.
- **Step 4:** Repeat until no new frequent itemsets are found.
- **Step 5:** Generate association rules from the frequent itemsets that meet minimum confidence.

Pseudocode (from Wikipedia ¹³):

```

Apriori(T,  $\epsilon$ ):
    L1 = {frequent 1-itemsets}
    k = 2
    while L(k-1)  $\neq \emptyset$ :
        Ck = GenerateCandidates(L(k-1), k)
        for each transaction t in T:
            for each candidate c in Ck:
                if c  $\subseteq$  t, then count[c]++
        Lk = {c  $\in$  Ck | count[c]  $\geq \epsilon$ }
        k = k + 1
    return  $\bigcup_k L_k$ 

```

(Here T = transaction database, ϵ = support threshold. `GenerateCandidates` joins $L(k-1)$ with itself and prunes any candidate with an infrequent subset ¹⁴.)

Apriori “grows” large itemsets one item at a time ¹¹. It is widely used in market-basket analysis to find items frequently bought together. From the frequent itemsets, one can derive rules like $X \Rightarrow Y$ (e.g. bread $\Rightarrow$ butter) that meet confidence thresholds. Apriori is well-known but can be slow if many candidates are generated ¹⁴.

Q2 (a) Cluster Analysis (Definition, Usage, Example)

Cluster analysis (clustering) is an unsupervised data mining task of grouping similar objects together so that objects in the same cluster are more similar to each other than to those in other clusters. In clustering, the data are unlabeled and the algorithm discovers groups based on features ⁶. Common clustering uses include identifying patterns or segments in data – for example, grouping customers by buying behavior, grouping documents by topic, or detecting anomalies.

A popular method is **K-means** (discussed above), which partitions data into k clusters by minimizing within-cluster variance ⁸. Other methods include hierarchical clustering (building a tree of clusters) and density-based clustering.

Example & Applications: Suppose a retail business wants to segment its customers. Using clustering (e.g. K-means on features like purchase history and demographics), the business might find groups such as “budget buyers” vs. “premium buyers.” This enables targeted marketing. In image analysis, clustering groups pixels to segment an image (e.g. separating background from object). In summary, clustering helps discover natural groupings in data ¹⁵.

Applications: Market segmentation, image segmentation, biological classification (gene expression grouping), document clustering for search, anomaly detection for fraud (finding outlier clusters), etc

¹⁵.

Q2 (b) Web Mining: Content and Usage

Web mining applies data mining techniques to Web data. Broadly it is divided into three categories ¹⁶ :

- **Web Content Mining**: extracting information from the *content* of web documents (text, images, audio, video). For example, text analysis on pages, or mining product descriptions for patterns. It is akin to text mining of web pages ¹⁷ .
- **Web Structure Mining**: analyzing the *link structure* of the Web (e.g. hyperlinks graph) to discover relationships between pages (e.g. PageRank or community detection).
- **Web Usage Mining**: analyzing *user access logs* to discover browsing patterns and user behavior. This involves mining server logs, clickstreams, cookies to find patterns (e.g. frequent navigation paths) ¹⁸ .

Tasks in Web Mining include content classification, information extraction, spam filtering (content tasks), link analysis, site structure summarization (structure tasks), and session/clustering analysis of user logs (usage tasks) ¹⁷ ¹⁸ . For instance, web usage mining can identify that many users navigate Product→Reviews pages, suggesting a redesign.

Q2 (c) Rule-Based Classification (Definition & Example)

A **rule-based classifier** uses a set of *if-then* rules to assign class labels. Each rule has an antecedent (conditions) and a consequent (class). For example ¹⁹ :

Rule R1: *IF age = youth AND student = yes THEN buys_computer = yes.*

Here, “age = youth $\wedge$ student = yes” is the antecedent, and “buys_computer = yes” is the consequent ¹⁹ . To classify a new instance, the classifier checks which rule antecedents are satisfied and applies the corresponding class.

Rule-based classifiers are often derived from decision trees (each path yields a rule) or through separate rule-induction algorithms. They are interpretable since the logic is explicit. In the above example, the rule predicts that young students buy computers. Multiple rules can cover different cases, and conflicts can be resolved by rule order or voting.

Q2 (d) Building a Classification Model (Steps & Example)

Key steps to build a classification model ²⁰ ²¹ :

1. **Data Collection**: Gather relevant labeled data for the problem (e.g. historical data of past cases with known classes). Ensure it has all necessary features and correct labels ²⁰ .
2. **Data Preprocessing**: Clean the data – handle missing values, remove inconsistencies, normalize or scale features. Convert categorical data to numerical if needed ²² .
3. **Feature Selection/Engineering**: Select or create the most informative attributes. Remove redundant features (e.g. using correlation analysis or PCA) ²³ .
4. **Choose Algorithm**: Select a suitable classifier (e.g. decision tree, SVM, neural network) based on data and task.
5. **Model Training**: Split data into training and test sets. Train the classifier on the training set (learn patterns) ²¹ .
6. **Model Evaluation**: Evaluate performance on the test set (metrics like accuracy, precision, recall). Tune the model (e.g. adjust parameters or features) as needed.

Example: To build a spam email classifier, we first collect a dataset of emails labeled “spam” or “not spam.” We preprocess (e.g. remove stopwords), select features (word frequencies, metadata), and choose a model (e.g. Naïve Bayes). We train on a subset of emails, then test accuracy on held-out data.

The resulting model can then predict the class (spam/ham) of new emails with measured accuracy ²⁰ ²¹ .

December 2023 Term-End Examination

Q1 (a) Decision Tree (Definition, Example, Strengths/Weaknesses)

A **Decision Tree** is a flowchart-like classification model where internal nodes test an attribute and leaf nodes represent class labels. For example, a tree might first split by “*Credit Score > 700?*”, then by “*Debt/Income Ratio < 0.3?*”, ultimately classifying loan applicants as “*Approved*” or “*Rejected*”. Decision trees are built by recursively partitioning the data (e.g. using information gain or Gini index to choose splits).

Strengths: Decision trees are easy to interpret (rules can be extracted), handle both numerical and categorical data, and require little data preprocessing. They capture non-linear relationships and can model complex decision boundaries.

Weaknesses: They can overfit if grown too deep. They are unstable (small data changes can yield different trees) and bias towards attributes with more levels. Complex trees are hard to interpret. Pruning is often needed to generalize well.

(A simple example: classify an animal as “Mammal” if fur=yes else “Reptile” if lays_eggs=yes, etc.)

Q1 (b) Data Lake Stages (Puddle, Pond, Lake, Ocean)

According to Alex Gorelik’s terminology ²⁴ ²⁵ :

- **Data Puddle:** A small, purpose-specific data mart built on big data technology for a single team or project. It contains a narrow, well-defined data scope and lowers IT cost for that use case ²⁴ .
- **Data Pond:** A collection of data puddles or a partial warehouse offloaded to big-data tech. It is like a big data-based warehouse but still limited in scope and requires IT support ²⁶ .
- **Data Lake:** A centralized repository of raw data that supports broad self-service analytics. It allows users to access and analyze any data they might need (even without a predefined project) ²⁵ .
- **Data Ocean:** Expands the lake further to include *all* enterprise data. It provides self-service access to data wherever it resides, with minimal IT involvement ²⁵ .

As maturity grows from puddle→pond→lake→ocean, data volume and user base increase dramatically, moving from high IT control to fully democratized access ²⁵ .

Q1 (c) Web Mining: Definition & Tasks

Web mining is the application of data mining techniques to discover information from Web data ²⁷ . It involves extracting patterns from web content, structure, and usage. Major tasks include: mining **web content** (e.g. extracting keywords/themes from page text, sentiment from reviews), mining **web structure** (analyzing hyperlinks and network graph), and mining **web usage** (finding browsing patterns from clickstream logs) ¹⁷ ¹⁸ .

Web mining tasks may include clustering similar web pages, classifying content categories, discovering frequent navigation paths, and extracting user profiles from log data. For example, mining web usage logs can identify that “50% of visitors who viewed Page A also viewed Page B,” aiding site optimization.

Q1 (d) OLAP (MOLAP, ROLAP, HOLAP)

OLAP (Online Analytical Processing) systems support fast, multidimensional queries. There are three main types:

- **MOLAP (Multidimensional OLAP):** Data is stored in optimized multidimensional cube structures. Query performance is fast due to pre-aggregation, but it can struggle with very large dimensionality or sparse data.
- **ROLAP (Relational OLAP):** Uses relational databases to store data and creates dynamic SQL queries. It can handle large data volumes and complex schemas (e.g. star/snowflake), but query performance may be slower than MOLAP because aggregations are computed on-the-fly.
- **HOLAP (Hybrid OLAP):** Combines MOLAP and ROLAP: summary data is stored in cubes (like MOLAP) while detailed data remains in relational tables. This provides a balance of storage efficiency and performance.

Each approach trades off query speed vs. storage/flexibility.

Q1 (e) OLTP vs OLAP

- **OLTP (Online Transaction Processing)** systems handle routine transaction-oriented operations (e.g. insert/update database records). Characteristics: many short on-line transactions, highly normalized schema, low query complexity, emphasis on data integrity and concurrency.
- **OLAP (Online Analytical Processing)** systems support complex queries and analysis (e.g. data warehousing). Characteristics: fewer transactions, complex queries aggregating large datasets, denormalized schemas (star/snowflake), emphasis on fast query performance and historical data.

In summary, OLTP is for day-to-day operations (e.g. banking transactions), whereas OLAP is for analytical queries on historical data (e.g. trend analysis).

References

- IBM Research – *K-means Clustering*: Overview and properties of k-means ⁶ ⁸ .
- Wikipedia – *Apriori Algorithm*: Description and pseudocode for frequent itemset mining ¹¹ ¹³ .
- Alex Gorelik (via Eryurek et al.) – *Data Lake Maturity*: Definitions of Data Puddle, Pond, Lake, Ocean ²⁴ ²⁵ .
- GeeksforGeeks – *Binning Method for Data Smoothing*: Explanation of binning (equal-frequency) ⁴ ⁵ .
- Tutorialspoint – *Rule-Based Classification*: IF-THEN rule example ¹⁹ .
- GeeksforGeeks – *Web Mining*: Definition and categories (content, structure, usage) ¹⁶ ¹⁷ .
- GeeksforGeeks – *Basic Classification*: Steps to build a classification model ²⁰ ²¹ .
- GeeksforGeeks – *Cluster Analysis*: Applications of clustering ¹⁵ .
- ThirdEye Data – *Hadoop DW Architecture*: Description of Hadoop DW components ¹ ³ .
- Additional definitions from data mining literature as cited above.

¹ ² ³ Hadoop Data Warehouse Project | Hadoop based data warehouse
<https://thirdeyedata.ai/apache-hadoop-based-datawarehousing/>

⁴ ⁵ Python | Binning method for data smoothing - GeeksforGeeks
<https://www.geeksforgeeks.org/machine-learning/python-binning-method-for-data-smoothing/>

6 7 8 9 **What is k-means clustering? | IBM**

<https://www.ibm.com/think/topics/k-means-clustering>

10 **k-means clustering - Wikipedia**

https://en.wikipedia.org/wiki/K-means_clustering

11 12 13 14 **Apriori algorithm - Wikipedia**

https://en.wikipedia.org/wiki/Apriori_algorithm

15 **Data Mining - Cluster Analysis - GeeksforGeeks**

<https://www.geeksforgeeks.org/data-analysis/data-mining-cluster-analysis/>

16 17 18 27 **Web Mining - GeeksforGeeks**

<https://www.geeksforgeeks.org/machine-learning/web-mining/>

19 **Randomized Base Classifiers in Data Mining**

https://www.tutorialspoint.com/data_mining/dm_rbc.htm

20 21 22 23 **Basic Concept of Classification (Data Mining) - GeeksforGeeks**

<https://www.geeksforgeeks.org/machine-learning/basic-concept-classification-data-mining/>

24 25 26 **Data Governance in the Cloud: Best Practices and Risk Management - Studocu**

<https://www.studocu.com/en-ca/document/fanshawe-college/information-security/data-governance-in-the-cloud-best-practices-and-risk-management/131138763>